KDG — Why the model under-performs, and a one-day plan to fix it

Scope: COMPS + IT syllabus data only (cross-domain is out of scope until there is more data). **Goal:** Upload a COMPS/IT syllabus → get clean topic nodes → get believable prerequisite edges, close to what a Gemini prompt gives. **Nothing in this document requires code to be read; it is a set of decisions and steps.** Evidence comes from inspecting `True_Cleaned_Dataset/final_training_dataset.csv`, `app/ml/`, `app/parser/`, `app/services/` and `DATA/extracted/`.

---

1. TL;DR

The model is **not** hallucinating topics. It cannot — it never reads a syllabus and never creates a topic. It only scores *pairs of topics that someone else already extracted*. The bad output has **two separate causes**:

Symptom you see	Real cause
Garbage nodes ("Page 2", "Official university", "This is a student")	**Extraction**, not ML. The live app uses a basic regex parser. The cleaning filters live only in an offline script.
Edges look like one long chain / wrong prerequisites	**Training labels were produced by heuristic rules**, so the model just learned "similar-sounding topics that are close together in the document are prerequisites".
Great metrics (F1 0.97) but poor real results	The test set is ~140 rows; the labels are circular; train and serve compute a feature differently.

The fix in one sentence: use Gemini as the *teacher* — to extract clean topics and to write prerequisite labels for COMPS/IT — then retrain the small local model on those labels with better features, and evaluate it on a hand-checked gold set that was never used for training.

---

2. Root causes (with evidence)

RC1 — Garbage nodes come from the extractor, not the model

- The upload endpoint builds nodes with `app/parser/syllabus_extractor.py` (a 234-line regex parser).
- The noise filters (publisher names, author lines, "Page N", exam-scheme text, 15-word cap) exist only in `scripts/pipeline/full_pipeline.py` and were never ported to the app.
- Your 2019 COMPS PDFs show the same problem in `DATA/extracted/`: of 592 topics across the five 2019 Computer Engineering files, **241 (41%) look malformed** (e.g. Final Year 7–8: 76 of 194; Sem 3: 58 of 154). The 2024-25 files extracted 0 topics at all.
- **Why it matters:** a clean model fed junk nodes outputs junk. Fixing the classifier alone cannot fix this.

RC2 — The training labels are circular (the biggest ML problem)

- `final_training_dataset.csv` has **1,171 rows, only 220 positive (19%)**.
- The `reason` column shows how labels were created: 125 of the 220 positives carry rule text such as "sim ≥ 0.55 + delta ≤ 5", "Adjacent same-course topics" or "Foundational keyword in A". Another 89 positives have no reason (provenance unknown). **705 negatives are simply "Insufficient signal".**
- `order_delta` almost decides the label by itself: negative deltas are **never** positive (0 of 35), delta ≥ 4 is positive only **3.4%** of the time (562 rows), delta = 0 is positive 95% of the time.
- **Why it matters:** the model learns to reproduce the rule, not real prerequisite knowledge. That is exactly why graphs come out as chains of neighbouring topics ("Pointers" needs 20 earlier topics).

RC3 — The features are too shallow for the task

- The model that the app actually loads (`model_artifact_v3.joblib`) uses only **3 features**: embedding similarity (importance 0.50), `order_delta` (0.39), keyword overlap (0.11).
- `domain_match` is **1.0 in every row** — it carries no information and is already dropped at serve time.
- Cosine similarity is **symmetric**; a prerequisite is **directional** (Arrays → Pointers, not the reverse). Direction is currently forced by "earlier unit/position wins", not learned.
- The model never sees *what the topics mean*, only how similar they are.

RC4 — Training and serving disagree

- Training: `order_delta` = position gap between two topics inside a subject (can be 0…dozens, even negative).
- Serving (old path): `order_delta` = absolute difference of **unit numbers** (0–5).
- Serving (old path) also computes embedding similarity from `name + unit + description`, but training used names only.
- The feature list in `metrics_report.json` / docs (4 features, `model_artifact.joblib`) does not match what the app loads (3 features, `v3`).
- **No script in the repo produces** `model_artifact_v3.joblib`, so nobody can reproduce or audit the deployed model.

RC5 — The data is tiny, mixed, and unevenly clean

- Only **322 unique concepts** and 12 source files. Training on so few concepts cannot generalise to new syllabi.
- 378 of 1,171 rows are **not COMPS/IT** (Chemical Eng, EM, EXTC and an "ASCEND seed" layer). Only about 793 rows (174 positives) are on-target.
- Leftover junk in the dataset: header text ("Course Code Course Name"), "Self learning topics", broken encoding ( ), merged multi-topic strings.
- The test split is ~140 rows, so "F1 0.97" means almost nothing. Cross-domain recall was 0.16.

RC6 — The expectation gap ("like Gemini")

Gemini reads the whole document and brings world knowledge about computing. A small tree model on three numbers cannot match that. The realistic way to get Gemini-like behaviour by tomorrow is **to let Gemini produce the knowledge (offline), and teach the local model from it**.

---

3. Target design

```
PDF --> (A) Gemini topic extraction --> clean topics per subject (cached JSON)
                                              |
                    (B) Gemini listwise labelling --> teacher edges (A→B)
                                              |
                    (C) Retrain local model on teacher labels (richer features)
                                              |
Upload > clean topics > (C) scores pairs > (D) graph rules > map / order / missing prerequisites
```

- **(A) Topics:** Gemini reads each subject and returns clean topic names. Already prototyped in the study-planner page; here it is also used offline to build the training topics. The regex parser stays only as a fallback.
- **(B) Labels:** Gemini is shown the **whole ordered topic list of one subject** and asked for each topic's *direct* prerequisites. (Listwise is cheaper and more consistent than asking about every pair.)
- **(C) Student model:** the local XGBoost (or logistic regression) is retrained on these labels, so the final product still has a *trained local ML model*. This keeps the project description honest.
- **(D) Deterministic graph rules:** keep it a DAG, limit parents per topic, remove redundant edges, make sure every non-root topic has a parent.

Fallback (Plan B, if labelling is not done in time): at upload time, ask Gemini for the edges directly, and use the local model only to re-rank/filter them. Quality is higher immediately, but say so in your report: the edges come

from an LLM, validated by the trained classifier.

---

4. Step-by-step instructions (one working day)

Times assume 2–3 people working in parallel. Work in this order.

Step 0 — Freeze scope and measure the baseline (≈ 45 min)

1. Decide the training universe: **Computer Engineering (2019 scheme, 6 PDFs), IT (6.34), 6.24 Comp Eng III–IV, CSE (6.54), CSE-AIML, AI-DS, AI-ML, DS, and SEM-V.** Everything else (Chemical, EM, EXTC, communication, chemistry) is *excluded from training*.
2. Keep the excluded rows only as a "different domain" test later; never train on them now.
3. Write down the current model's numbers on the new gold set (Step 5) so you can *prove* improvement.

Step 1 — Extract clean topics with Gemini for every COMPS/IT PDF (≈ 2 h)

1. For each PDF, first ask Gemini to list the subjects (code + title). Skip labs, mini-projects, workshops and capstones (they have no teachable topics).
2. For each subject, ask Gemini for: course code, title, semester, and **units → topics** (one concept per topic).
3. Prompt rules to include (in plain English):
   - topic = a short concept name, 2–8 words;
   - split comma-separated lists into separate topics; merge trivial sub-points;
   - **exclude** cover text, headers, page numbers, marks, hours, credits, outcomes, references, books, authors, publishers, "self-learning" boilerplate;
   - keep the syllabus order; temperature 0; JSON-only output.
4. Save one JSON per subject (cache), so you never pay for the same call twice.
5. **Automatic quality gate** on every subject — reject and re-run if any fails:
   - 8–60 topics per subject;
   - no topic longer than 8 words or containing marks/credits/reference/course-code words;
   - no duplicate names; no non-ASCII garbage;
   - topic count roughly matches the number of units × 3–6.
6. Manually spot-check **5 random subjects** (15 min). Target: garbage-node rate under 2%.

Why: this removes RC1 for training data and gives you the same clean topics the app will see at runtime.

Step 2 — Generate teacher labels with Gemini (≈ 2 h, mostly waiting)

1. For each subject, send the ordered topic list (with ids) and ask, **per topic**, for:
   - up to 3 *direct* prerequisites chosen **only from this subject's list** (by id);
   - optionally, outside concepts that are needed but not in the list (these also feed "missing prerequisites").
2. Rules to put in the prompt: direct prerequisites only (no chains); a topic may have none; never point to a later topic unless truly required; no self-references.
3. **Self-consistency:** run each subject twice with the topic order shuffled. Keep an edge as a **confident positive** only if both runs agree; edges in only one run are **uncertain** (exclude them from training, use them for review).
4. Also run subjects of the **same semester together** for a small cross-subject pass (e.g. Discrete Maths → Data Structures), optional if time is short.
5. Expected volume: ~40 subjects × 25–45 topics ≈ 1,200–1,800 topics → roughly 2,000–3,000 confident positive edges. That is **10× more positives** than today.

Step 3 — Build the training table (≈ 1 h)

1. **Positives:** confident teacher edges.
2. **Negatives (3 : 1 to positives), choose deliberately:**
   - *reversed* edges (B→A of every positive) — teaches direction;

- *hard* negatives: highly similar pairs the teacher did **not** link;
- *adjacent* pairs the teacher did not link — breaks the "neighbour = prerequisite" shortcut;
- a smaller share of random same-subject pairs.

3 Remove every row that came from the old rule-based labels. **Do not reuse** `reason`**/"Insufficient signal" labels.**

4 Clean text: fix the encoding issue, lowercase/trim consistently, drop header-like strings.

Step 4 — Features (≈ 1 h; decide first, then implement)

Use **one feature function shared by training and the app** (RC4). Recommended set:

1 Embedding of A and embedding of B, plus their **difference** and **element-wise product** (gives direction + meaning, standard pair-classification recipe). If tree training is slow on ~1,500 columns, reduce each embedding to ~32 dimensions with PCA fitted on training data only.

2 Cosine similarity.

3 **Normalised position gap** = (position of B − position of A) ÷ number of topics in the subject; plus a **signed** unit gap. Never use the absolute unit difference.

4 Whether A is in an earlier unit than B (and same-unit flag).

5 Keyword overlap and "name of A appears inside name of B".

6 Foundational-word flag for A ("introduction", "basics", "fundamentals", …).

7 Drop `domain_match` (constant).

8 Embed **names only**, always, in both training and serving.

Optional stretch (only if everything else is done): try a stronger embedding model, or fine-tune a small cross-encoder on the pairs. Do not start this before the baseline pipeline is working end to end.

Step 5 — Splits and a gold test set (≈ 1 h; start early, run in parallel)

1 **Split by subject (normalised subject title), not by row and not by PDF.** Computer Engineering and IT repeat many subjects (DBMS, OS, DS…); if the same subject lands in both train and test the score is fake.

2 Hold out at least **one whole PDF/branch** (for example IT 6.34) as the test.

3 **Gold set:** hand-check **300 pairs** (150 positive, 150 negative) from test subjects only. Three people, 100 each, 30–40 min. The model must **never** train on these.

4 Also hand-check **150 random teacher edges** from training subjects to measure the teacher's own precision. If it is under ~80%, tighten the prompt and rerun Step 2 before training.

Step 6 — Train, calibrate, compare (≈ 1.5 h)

1 Train a logistic-regression baseline and an XGBoost model on the new table. Keep depth modest (about 4–6) with early stopping on the validation subjects.

2 Pick the decision threshold by maximising **graph-level** F1 on validation subjects, not row-level accuracy.

3 Evaluate the **old model and the new model** on the same gold set. Report precision, recall, F1 and PR-AUC.

4 Save the artefact **together with** its feature list, embedding model name, threshold and training-data hash (one file or one folder). The app must refuse to load a model whose feature list does not match.

5 Commit the training script that produced it (so `v3`-style mystery models cannot happen again).

Step 7 — Graph rules at serve time (≈ 45 min)

1 Candidate pairs: forward pairs in syllabus order inside the subject (plus the optional cross-subject set).

2 Keep an edge if probability ≥ threshold; allow **at most 2–3 parents per topic**.

3 Remove redundant edges (if A→B→C exists, drop A→C).

4 If a non-first topic has no parent, attach its single best-scoring candidate so the map has no stray nodes.

5 Compute the study order, key topics and missing prerequisites from this graph as today.

Step 8 — Wire it into the app and test (≈ 1 h)

1. Make the app's topic source the Gemini extraction (already in the study-planner page); keep the regex parser as fallback only.
2. Run 5 syllabi the model has never seen (different branches). Check them against the acceptance list below.

---

5. Acceptance checklist ("done" means all true)

Check	Target
Garbage nodes (header/page/watermark/author text)	**< 2%** of nodes on 5 unseen syllabi
Edge precision on the hand-checked gold set	**≥ 0.80**
Edge recall on the gold set	**≥ 0.70**
Beats the old model on the same gold set	**Yes**, by a clear margin
Graph is a valid DAG, avg parents per topic	**1–2.5**
Nodes connected to at least one edge	**≥ 90%**
Unseen-subject performance reported on ≥ 300 gold pairs	**Yes** (never on ~140 rows)
Model, features, threshold, data version stored together	**Yes**
Training script for the deployed model exists in the repo	**Yes**

---

6. Things to stop doing

- **Do not** train on the old rule-generated labels or the `reason` text.
- **Do not** use random row splits or evaluate on a ~140-row test set.
- **Do not** mix Chemical/EM/EXTC rows into COMPS/IT training.
- **Do not** feed regex-extracted topics into training or the live app.
- **Do not** compute a feature differently in training and in the app.
- **Do not** claim the model "learned prerequisites from data" without saying the labels were produced by an LLM teacher and checked by hand.

---

7. Risks and mitigations

Risk	Mitigation
Gemini free-tier rate limits / daily quota	Cache every response; run subjects in a queue with retries; start Steps 1–2 first thing; keep the model name configurable.
Teacher labels are noisy	Two shuffled runs + agreement filter; hand-check 150 edges; tighten the prompt if precision < 80%.
Train/test leakage (same subject in Comp and IT)	Group by normalised subject title; hold out a whole PDF.
Small data → overfitting	Depth-limited trees, early stopping, subject-wise validation, 10× more positives than today.
Time overrun	Work to Plan B (Gemini edges + local re-ranker) and disclose it; the retrain can finish afterwards.
Reviewers ask "where is the ML?"	The student model is trained locally on labelled pairs with reported subject-held-out metrics; the LLM is the labelling teacher, documented as such.

---

8. Suggested split of work (3 people)

- **Person 1:** Steps 1–2 (Gemini extraction + teacher labels, quality gate, caching).
- **Person 2:** Steps 3–4 and 6 (training table, shared feature function, training and comparison).
- **Person 3:** Step 5 (splits, gold set, review sheets) then Steps 7–8 (graph rules, app wiring, 5-syllabus test).

---

9. Quick reference — key numbers behind this plan

Fact	Value
Training rows / positives	1,171 / 220 (18.8%)
Unique concepts	322
Rows that are COMPS/IT	≈ 793 (174 positives); ≈ 378 are other domains
Positives created by heuristic rule text	125 of 220
Negatives labelled "Insufficient signal"	705 of 951
Positive rate when position gap ≥ 4	3.4% (562 rows)
Positive rate when position gap is negative	0% (35 rows)
Features in deployed model (v3)	3 (similarity 0.50, position 0.39, keyword 0.11)
`domain_match` values	all 1.0 (no information)
Suspect topics in 2019 Comp Eng extractions	241 of 592 (41%)
Held-out test split in the old metrics	~140 rows
Cross-domain recall of the old model	0.16